

Содержание

1. Глоссарий	4
1.1. API	4
1.2. BI	4
1.3. CDN	4
1.4. Компонент	4
1.5. CRM	4
1.6. СУБД	4
1.7. Индекс	4
1.8. Ввод-вывод	4
1.9. IoT	4
1.10. Слой	4
1.11. Метаданные	4
1.12. OLAP	5
1.13. OLTP	5
1.14. Сессия	5
1.15. SLA	5
1.16. Транзакция	5
2. Архитектура информационных систем	6
2.1. Модуль 1: Архитектурные основы	6
2.1.1. Лекция 1: Введение. Принципы построения приложения	6
2.1.1.1. 1. Компонент как класс (C++)	6
2.1.1.2. Зависимости компонентов и слои	7
2.1.1.3. Продуктивность слоя	8
2.1.1.4. Оптимизация системы требует выбора критерия оптимизации	9
2.1.1.5. Принципы SOLID	9
2.1.1.6. Dependency Injection	9
2.1.1.7. Clean Architecture	9
2.1.2. Лекция 2: Альтернативные архитектурные паттерны	10
2.1.2.1. Архитектура на основе фреймворка (Rails, Django)	10
2.1.2.2. Модель акторов (Erlang/Elixir/Akka)	10
2.1.2.3. Проектирование, ориентированное на данные (Data-oriented design)	10
2.1.3. Лекция 3: Протоколы взаимодействия: REST / gRPC	10
2.1.3.1. REST: стандартный веб-протокол	10
2.1.3.2. gRPC: бинарный протокол, высокая производительность	10
2.1.4. Лекция 4: Протоколы взаимодействия: GraphQL / WebSocket	11
2.1.4.1. GraphQL: клиент запрашивает только нужные данные	11
2.1.4.2. WebSocket: двусторонний канал	11
2.2. Модуль 2: Фундаментальные структуры хранилищ данных	12
2.2.1. Лекция 5: Классификация нагрузки	12
2.2.1.1. Типы операций	12
2.2.1.2. Метрики нагрузки	12
2.2.1.3. Характеристики нагрузки	12
2.2.1.4. Оценка QPS	13
2.2.1.5. Оценка объёма хранения	14
2.2.1.6. Иерархия памяти	14
2.2.1.7. Фундаментальные компромиссы	14
2.2.2. Лекция 6: LSM Дерево	15
2.2.2.1. Общая идея	15
2.2.2.2. Отсортированная таблица строк (Sorted String Table, SSTable)	15
2.2.2.3. Memtable	15
2.2.2.4. Bloom Filter	15

2.2.2.5. Компакция – причины, уровни	16
2.2.2.6. Критерии запуска компакций	16
2.2.2.7. Что происходит при компакции	17
2.2.2.8. Почему L_0 специальный?	17
2.2.2.9. Tombstones	17
2.2.2.10. Полная структура LSM-дерева	17
2.2.3. Лекция 7: B-дерево: индексирование на диске	19
2.2.3.1. B-фактор и структура узла	19
2.2.4. Лекция 8: ACID, уровни изоляции, аналитические и транзакционные хранилища	21
2.2.4.1. Транзакции	21
2.2.4.2. ACID (Atomicity, Consistency, Isolation, Durability)	21
2.2.4.3. Транзакционные и аналитические хранилища	21
2.2.4.4. Сжатие колонок	22
2.2.5. Лекция 9: Хранилища в оперативной памяти	23
2.2.5.1. Аналитика: DuckDB и векторизация	23
2.2.5.2. Кэширование: Redis	23
2.3. Модуль 3: Методы поиска похожих/близких элементов	24
2.3.1. Лекция 10: Специализированные индексы для поиска	24
2.3.1.1. Inverted Index (перевёрнутые индексы)	24
2.3.1.2. Trie (префиксные деревья)	24
2.3.1.3. Bitmap indexes	24
2.3.2. Лекция 11: Векторные индексы для поиска подобия	24
2.3.2.1. Основы векторных представлений	24
2.3.2.2. HNSW (Hierarchical Navigable Small World)	24
2.3.2.3. LSH (Locality-Sensitive Hashing)	24
2.3.2.4. Примеры: Weaviate, Pinecone, Qdrant	24
2.3.3. Лекция 12: Геопространственные структуры	24
2.3.3.1. R-Tree	24
2.3.3.2. QuadTree и KD-Tree	24
2.4. Модуль 4: Операционные компоненты	24
2.4.1. Лекция 13: Прокси и Rate Limiting	24
2.4.1.1. Forward Proxy / Reverse Proxy	24
2.4.1.2. Распределение нагрузки	24
2.4.1.3. Ограничение частоты запросов	24
2.4.2. Лекция 14: Безопасность и управление доступом	24
2.4.2.1. Аутентификация и авторизация	24
2.4.2.2. RBAC и ABAC	24
2.4.2.3. Single Sign-On	24
2.4.3. Лекция 15: Логирование, мониторинг и планирование задач	25
2.4.3.1. Сбор логов	25
2.4.3.2. Сбор метрик (Prometheus)	25
2.4.3.3. Мониторинг и диагностика (Grafana)	25
2.4.3.4. Планировщики (Celery, Airflow, Dagster)	25
2.5. Модуль 5: System Design	25
2.5.1. Лекция 16: Разбор дизайна конкретной системы	25
2.5.1.1. Применение всех концепций на практике	25
3. Распределённые системы обработки данных	26
3.1. Модуль 1: Компоненты распределённых систем	26
3.1.1. Лекция 1: Основы распределённых систем	26
3.1.1.1. CAP-теорема	26
3.1.1.2. Механизмы репликации данных	26
3.1.1.3. ZooKeeper: распределённое решение проблем согласованности и отказоустойчивости ..	

3.1.2. Лекция 2: Гарантии консистентности и координация распределённых транзакций	26
3.1.2.1. Алгоритмы достижения консенсуса	26
3.1.2.2. Модель конечной консистентности (Eventual consistency) и требование идемпотентности операций	26
3.1.2.3. Протокол двухфазного коммита (Two-Phase Commit, 2PC)	26
3.2. Модуль 2: Распределённые сервисы	26
3.2.1. Лекция 3: Асинхронное взаимодействие сервисов	26
3.2.1.1. Event Bus и Event-Driven Architecture	26
3.2.1.2. Publish-Subscribe vs Message Queue	26
3.2.1.3. Saga Pattern	26
3.2.2. Лекция 4: Kafka	26
3.2.2.1. Внутреннее устройство	26
3.2.3. Лекция 5: Kubernetes	26
3.2.3.1. Роль и место в современных архитектурах	26
3.2.3.2. Области использования	26
3.2.3.3. Внутреннее устройство	26
3.2.4. Лекция 6: ClickHouse	26
3.2.4.1. Архитектура системы хранения и обработки данных	26
3.2.4.2. Партиционирование	26
3.2.4.3. Таблицы серии MergeTree как основной механизм персистентности	27
3.2.5. Лекция 7: Apache Spark	27
3.2.5.1. Роль и место в современных архитектурах	27
3.2.5.2. Области использования	27
3.2.5.3. Внутреннее устройство	27
3.2.5.4. API	27
3.2.6. Лекция 8: Erlang	27
3.2.6.1. Роль и место в современных архитектурах	27
3.2.6.2. Erlang VM (BEAM) и процессы	27
3.2.6.3. Модель акторов	27
3.2.6.4. Fault tolerance и “Let it crash” философия	27
3.2.6.5. OTP фреймворк (Supervisor, GenServer)	27
3.2.6.6. Синхронизация и распределённые вычисления	27
3.2.6.7. Elixir	27
3.3. Модуль 3: Распределённые БД	27
3.3.1. Лекция 9: Большие данные	27
3.3.1.1. Свойства	27
3.3.1.2. Архитектуры	27
3.3.2. Лекция 10: Full Text Search в распределённых системах	27
3.3.2.1. Elasticsearch и Solr	27
3.3.2.2. Распределённый поиск	27
3.3.2.3. Масштабирование	27
3.3.3. Лекция 11: Документные хранилища	28
3.3.3.1. MongoDB: sharding strategies, write concerns	28
3.3.3.2. Object storage: MinIO	28
3.3.4. Лекция 12: Распределённые пространственные и графовые БД	28
3.3.4.1. Графовые БД	28
3.3.4.2. Пространственные БД	28
3.3.4.3. Шардирование графов и пространственных данных	28

1. Глоссарий

1.1. API

Application Programming Interface — программная абстракция, определяющая методы и протоколы взаимодействия между программными компонентами. Обеспечивает унифицированный доступ к функциональности системы посредством стандартизированных вызовов, скрывая детали внутренней реализации.

1.2. BI

Business Intelligence — технологии и практики сбора, интеграции, анализа и представления бизнес-информации. Включает инструменты для создания отчётов, дашбордов, визуализации данных и поддержки принятия управленческих решений на основе данных.

1.3. CDN

Content Delivery Network — географически распределённая сеть серверов для кэширования и доставки статического контента пользователям с минимальной задержкой.

1.4. Компонент

Единица вычисления либо хранения данных, имеющая явно определённый контракт взаимодействия (интерфейс) и набор зависимостей.

1.5. CRM

Customer Relationship Management — система управления взаимоотношениями с клиентами. Программное обеспечение для автоматизации стратегий взаимодействия с клиентами, включая продажи, маркетинг, техподдержку и аналитику клиентских данных.

1.6. СУБД

Система управления базами данных — комплекс программных средств для создания, модификации и управления данными в базе данных. Обеспечивает структурированное хранение, эффективный доступ, целостность данных и управление конкурентным доступом.

1.7. Индекс

Вспомогательная структура данных в системах управления базами данных, предназначенная для оптимизации производительности операций поиска и извлечения информации. Организует упорядоченное представление данных для существенного сокращения времени доступа к записям.

1.8. Ввод-вывод

Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

1.9. IoT

Internet of Things — сетевое взаимодействие физических устройств с датчиками и контроллерами. Характеризуется высокой частотой передачи небольших порций данных (телеметрия, метрики).

1.10. Слой

Множество компонентов, объединённых общей ответственностью и уровнем абстракции, и участвующих в системе согласно установленным правилам межслойных зависимостей. Говорят что слой А зависит от слоя Б если хотя бы один компонент слоя А зависит от компонента слоя Б.

1.11. Метаданные

Структурированная информация, описывающая свойства других данных: время создания, размер, тип, владелец, права доступа. Используется для индексирования и маршрутизации без чтения основного содержимого.

1.12. OLAP

Online Analytical Processing — класс систем для аналитической обработки данных. Ориентирован на выполнение сложных агрегирующих запросов по большим объёмам данных, использует колоночное хранение для оптимизации производительности аналитики.

1.13. OLTP

Online Transaction Processing — класс систем обработки транзакций в реальном времени. Характеризуется большим количеством коротких транзакций (чтение, вставка, обновление, удаление), строковым хранением данных и требованиями к консистентности.

1.14. Сессия

Логический контекст взаимодействия клиента с системой между множественными запросами. Хранит состояние пользователя: аутентификацию, корзину, настройки. Реализуется через cookies + серверное хранилище (Redis) или JWT-токены.

1.15. SLA

Service Level Agreement — соглашение об уровне обслуживания с количественными метриками: uptime (99.9% = 8.76 ч простоя/год), задержка (p99 < 200 мс).

1.16. Транзакция

Последовательность операций чтения и записи в базе данных, рассматриваемая как единая логическая единица работы. Гарантирует атомарность выполнения: либо все операции завершаются успешно и фиксируются (COMMIT), либо при ошибке все изменения откатываются (ROLLBACK).

2. Архитектура информационных систем

2.1. Модуль 1: Архитектурные основы

2.1.1. Лекция 1: Введение. Принципы построения приложения

Компонент — Единица вычисления либо хранения данных, имеющая явно определённый контракт взаимодействия (интерфейс) и набор зависимостей.

Рассмотрим несколько примеров:

2.1.1.1. 1. Компонент как класс (C++)

```
#include <iostream>
```

```
class NumberPrinter {
public:
    void print(int x) {
        std::cout << x << std::endl;
    }

    void printRange(int from, int to) {
        for (int i = from; i <= to; ++i) {
            std::cout << i << std::endl;
        }
    }
};
```

Единица вычисления: форматирование и вывод чисел

Интерфейс: сигнатуры публичных методов

Зависимость: консоль ввода-вывода

Форма реализации: класс (ООП-модель)

- Компонент как функция (Elixir)

```
def send_message(host, port, message) do
  {:ok, socket} = :gen_tcp.connect(host, port, [:binary, active: false])
  :ok = :gen_tcp.send(socket, message)
  :gen_tcp.close(socket)
end
```

Единица вычисления: передача сообщения

Интерфейс: сигнатура функции

Зависимость: сеть (TCP stack)

Форма реализации: функция (функциональная модель)

- Компонент как функция (Go)

```
func LoadUserNames(db *sql.DB) ([]string, error) {
    rows, err := db.Query("SELECT name FROM users")
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var result []string
    for rows.Next() {
        var name string
        if err := rows.Scan(&name); err != nil {
            return nil, err
        }
    }
}
```

```

    result = append(result, name)
}
return result, nil
}

```

Единица вычисления: извлечение данных

Интерфейс: сигнатура функции

Зависимость: БД (SQL, соединение)

Форма реализации: функция (процедурная / Go-модель)

2.1.1.2. Зависимости компонентов и слои

Слой — Множество компонентов, объединённых общей ответственностью и уровнем абстракции, и участвующих в системе согласно установленным правилам межслойных зависимостей. Говорят что слой А зависит от слоя Б если хотя бы один компонент слоя А зависит от компонента слоя Б.

Рассмотрим пример слоя персистентности, который сохраняет различные модели в CSV:

```

// domain/models.go - слой моделей данных
package domain

type User    struct{ ID int; Name, Email string }
type Item    struct{ ID int; Title string; Price float64 }
type Storage struct{ ID int; Location string; Capacity int }

// persistence/csv_persistence.go - слой персистентности
package persistence

import (
    "fmt"
    "os"
    "strings"

    "example/domain" // Явная зависимость от слоя моделей
)

func SaveUsersToCSV(users []domain.User, filename string) error {
    lines := make([]string, 0, len(users))
    for _, u := range users {
        lines = append(lines, fmt.Sprintf("%d,%s,%s", u.ID, u.Name, u.Email))
    }
    return write(filename, lines)
}

func SaveItemsToCSV(items []domain.Item, filename string) error {
    lines := make([]string, 0, len(items))
    for _, i := range items {
        lines = append(lines, fmt.Sprintf("%d,%s,%v", i.ID, i.Title, i.Price))
    }
    return write(filename, lines)
}

func SaveStoragesToCSV(storages []domain.Storage, filename string) error {
    lines := make([]string, 0, len(storages))
    for _, s := range storages {
        lines = append(lines, fmt.Sprintf("%d,%s,%d", s.ID, s.Location, s.Capacity))
    }
    return write(filename, lines)
}

func write(filename string, lines []string) error {

```

```

return os.WriteFile(filename, []byte(strings.Join(lines, "\n")), 0644)
}

```

Анализ слоёв:

Слой	Компонент	Контракт (интерфейс)	Зависимости
domain	User	struct{ ID int; Name, Email string }	—
	Item	struct{ ID int; Title string; Price float64 }	—
	Storage	struct{ ID int; Location string; Capacity int }	—
persistence	SaveUsersToCSV	([]domain.User, string) error	domain.User, ФС
	SaveItemsToCSV	([]domain.Item, string) error	domain.Item, ФС
	SaveStoragesToCSV	([]domain.Storage, string) error	domain.Storage, ФС
	write	(string, []string) error	ФС

Заметим:

- **Слой моделей** (domain): три компонента-структуры без зависимостей
- **Слой персистентности** (persistence): четыре компонента-функции, три из которых явно зависят от типов слоя моделей
- Слой персистентности **зависит** от слоя моделей, но не наоборот

2.1.1.3. Продуктивность слоя

Прокси-компонент — компонент, не реализующий собственной вычислительной ответственности, а лишь делегирующий вызовы компонентам других слоёв.

Продуктивность слоя — количественная мера архитектурной самостоятельности слоя, определяемая долей компонентов, реализующих собственную вычислительную ответственность.

Формальное задание меры

Пусть:

- N — общее количество компонентов в слое
- P — количество прокси-компонентов в слое

Тогда коэффициент проксирования слоя определяется как:

$$K_{\text{proxy}} = \frac{P}{N}$$

а продуктивность слоя — как дополняющая величина:

$$K_{\text{prod}} = 1 - K_{\text{proxy}} = 1 - \frac{P}{N}$$

В стандартной литературе (например, Clean Architecture) слой считается продуктивным, если $K_{\text{prod}} \geq 0.8$, то есть не менее 80% компонентов реализуют собственную вычислительную ответственность.

Пример непродуктивного pass-through слоя:

Рассмотрим излишний слой сервисов, большинство компонентов которого только делегируют вызовы к слою персистентности:

```

// service/user_service.go - непродуктивный слой сервисов
package service

import (
    "example/domain"

```



```

    "example/persistence"
    "strings"
)

func GetUser(id int) (*domain.User, error) {
    return persistence.LoadUser(id)
}

func SaveUser(user *domain.User) error {
    return persistence.SaveUser(user)
}

func DeleteUser(id int) error {
    return persistence.DeleteUser(id)
}

func ListUsers() ([]*domain.User, error) {
    return persistence.ListUsers()
}

func SearchUsersByName(query string) ([]*domain.User, error) {
    users, err := persistence.ListUsers()
    if err != nil {
        return nil, err
    }

    var result []*domain.User
    for _, u := range users {
        if strings.Contains(strings.ToLower(u.Name), strings.ToLower(query)) {
            result = append(result, u)
        }
    }
    return result, nil
}

```

Анализ слоя сервисов:

Компонент	Контракт	Собственная ответственность
GetUser	(int) (*User, error)	Нет — прямая делегация
SaveUser	(*User) error	Нет — прямая делегация
DeleteUser	(int) error	Нет — прямая делегация
ListUsers	() ([]*User, error)	Нет — прямая делегация
SearchUsersByName	(string) ([]*User, error)	Да — фильтрация и поиск

Продуктивность слоя: $K_{\text{prod}} = 1 - \frac{4}{5} = 0.2$ (20% — непродуктивный слой)

Такой слой не добавляет архитектурной ценности и лишь усложняет кодовую базу, увеличивая количество уровней косвенности без предоставления дополнительной функциональности.

2.1.1.4. Оптимизация системы требует выбора критерия оптимизации

2.1.1.5. Принципы SOLID

2.1.1.6. Dependency Injection

2.1.1.7. Clean Architecture

2.1.2. Лекция 2: Альтернативные архитектурные паттерны

2.1.2.1. Архитектура на основе фреймворка (Rails, Django)

Фреймворк предполагает стандартную структуру папок, именование файлов и шаблоны проектирования (зачастую, MVC), что уменьшает количество принимаемых решений и ускоряет разработку.

- “Мнения” фреймворка: Rails/Django имеют предустановленные “мнения” по работе с БД (ORM), маршрутизацией, шаблонизацией и аутентификацией.

2.1.2.2. Модель акторов (Erlang/Elixir/Akka)

- Революционная технология, созданная в Ericsson еще в 1986 году для телекоммуникационных систем с требованием “пяти девяток” доступности.
- Она на десятилетия опередила современные подходы: предоставляет
 - встроенную распределенность (кластеры из коробки),
 - философия “Let it Crash!” с автоматическим самовосстановлением
 - горячее обновление кода без остановки системы
 - невероятную параллельность (миллионы изолированных процессов)
- WhatsApp: Всего 50 инженеров обрабатывали 2 миллиарда сообщений в день на одном сервере!
- Discord

2.1.2.3. Проектирование, ориентированное на данные (Data-oriented design)

Подход, фокусирующийся на эффективной организации данных в памяти для оптимального использования кэша процессора. Используется где производительность упирается в скорость доступа к памяти.

- игровые движки
- обработки физики
- AI
- симуляции

2.1.3. Лекция 3: Протоколы взаимодействия: REST / gRPC

API¹

2.1.3.1. REST: стандартный веб-протокол

- HTTP
 - методы (GET, POST, PUT, DELETE)
 - коды (Forbidden, Not Found)
- JSON
- URI Endpoints (например, /api/users/123)
- Swagger
- OpenAPI
 - Генерация спецификации из кода
 - Генерация кода из спецификации

2.1.3.2. gRPC: бинарный протокол, высокая производительность

- Историческая справка
 - Вырос из внутренней технологии Google под названием Stubby
 - Использовался для связи между тысячами микросервисов внутри дата-центров
 - В 2015 году открыли исходный код
- IDL (Описание формата общения в .proto-файлах)
- Генерация кода
- Мультиплексирование потоков (Stream Multiplexing)
- Двухнаправленная потоковая передача
- gRPC-Web прокси

¹Application Programming Interface — программная абстракция, определяющая методы и протоколы взаимодействия между программными компонентами. Обеспечивает унифицированный доступ к функциональности системы посредством стандартизированных вызовов, скрывая детали внутренней реализации.

2.1.4. Лекция 4: Протоколы взаимодействия: GraphQL / WebSocket

2.1.4.1. GraphQL: клиент запрашивает только нужные данные

Клиент формирует запрос с деревом полей, которые он хочет получить

- Проблема необходимости написания десятков различных запросов для получения одной сущности в различных экранах
- Подписка на реальные обновления данных
- Единственная конечная точка (Endpoint)

2.1.4.2. WebSocket: двусторонний канал

- Проблемы Long Polling
- Постоянное соединение, обмен сообщениями в реальном времени в обе стороны.
- Применение
 - Чат-приложения и онлайн-игры.
 - Биржевые тикеры и спортивные результаты.
 - Совместное редактирование документов.
 - Уведомления в реальном времени.

2.2. Модуль 2: Фундаментальные структуры хранилищ данных

Мы познакомились с архитектурными паттернами построения приложений и протоколами обмена данными между компонентами. Теперь возникает следующий вопрос: где и как хранить данные?

Задача выбора хранилища нетривиальна. На рынке сотни решений: реляционные базы, документные хранилища, key-value stores, колоночные базы, графовые базы, временные ряды. Каждое решение оптимизировано под определённый паттерн нагрузки, и неправильный выбор приводит к проблемам производительности, которые сложно исправить без миграции.

Чтобы осознанно выбирать между PostgreSQL и ClickHouse, между Redis и MongoDB, нужно понимать внутреннее устройство этих систем. В этом модуле мы разберём фундаментальные структуры данных, лежащие в основе современных хранилищ.

2.2.1. Лекция 5: Классификация нагрузки

При проектировании реальных систем первый шаг — оценка нагрузки. Без конкретных цифр невозможно принять обоснованное решение о выборе хранилища.

2.2.1.1. Типы операций

Зачастую взаимодействие с хранилищем сводится к ограниченному набору операций:

Тип	Операция	Пример
Чтение	Point lookup	SELECT * FROM users WHERE id = 123
	Multi-get	SELECT * FROM users WHERE id IN (1, 2, 3)
	Range scan	SELECT * FROM orders WHERE date BETWEEN '...' AND '...'
	Prefix scan	SELECT * FROM logs WHERE key LIKE 'user:123: %'
	Full scan	SELECT AVG(price) FROM products
Запись	Insert	вставка новой записи (ошибка при дубликате)
	Update	изменение существующей записи
	Upsert	INSERT ... ON CONFLICT DO UPDATE
	Delete	удаление записи
	Batch write	запись нескольких записей
Комб.	Read-modify-write	UPDATE accounts SET balance = balance + 100
	Increment	INCR views:page:123 (атомарный счётчик)

2.2.1.2. Метрики нагрузки

Метрика	Расшифровка	Описание
DAU	Daily Active Users	уникальные пользователи за сутки
MAU	Monthly Active Users	уникальные пользователи за месяц
RPS	Requests Per Second	пользовательских запросов (например HTTP) в секунду
QPS	Queries Per Second	запросов к БД в секунду
TPS	Transactions Per Second	транзакций БД в секунду
IOPS	I/O Operations Per Second	операций ввода-вывода в секунду

2.2.1.3. Характеристики нагрузки

Read/Write ratio:

- **90/10** — преобладание чтения: кэширование, CDN², статический контент
- **50/50** — сбалансированная: социальные сети, e-commerce
- **10/90** — преобладание записи: логирование, IoT³, метрики

²Content Delivery Network — географически распределённая сеть серверов для кэширования и доставки статического контента пользователям с минимальной задержкой.

Размер записи:

- < 1 KB — Метаданные⁴, Сессия⁵, счётчики
- 1-100 KB — JSON-документы, профили пользователей
- 100 KB - 1 MB — статьи, посты с изображениями
- > 1 MB — медиафайлы, бинарные данные

Паттерн доступа:

- **random** — point lookup по ключу, случайные запросы
- **sequential** — range scan, time-series, логи

Требования к латентности:

- **p50 < 50 мс** — медиана, типичный пользовательский опыт
- **p99 < 200 мс** — 1% худших запросов, важно для SLA⁶
- **p999 < 1 с** — 0.1% худших, критично для финтеха

Почему p99 важнее среднего: если p99 = 500 мс при 10000 QPS, то 100 запросов в секунду будут медленными — это 8.6 млн медленных запросов в сутки.

2.2.1.4. Оценка QPS

Пример расчёта — социальная сеть:

- 100 млн DAU (Daily Active Users)
- 300 млн MAU (Monthly Active Users)
- Каждый пользователь в день:
 - 10 просмотров ленты (каждый = 1 HTTP запрос к API)
 - 2 лайка
 - 0.1 поста
 - 5 загрузок изображений профилей

Полезные константы: секунд в сутках $\approx 10^5$, в месяце $\approx 2.5 \times 10^6$, в году $\approx 3 \times 10^7$.

RPS (Requests Per Second) — пользовательские HTTP-запросы:

$$\text{RPS} = \frac{100 \times 10^6 \times 10}{10^5} \approx 10000 \text{ RPS}$$

QPS (Queries Per Second) — запросы к БД:

- Просмотр ленты: 1 запрос на выборку постов + 1 на подсчёт лайков = 2 запроса к БД
- Лайк: 1 INSERT + 1 UPDATE счётчика = 2 запроса к БД

$$\text{Read QPS} = \frac{100 \times 10^6 \times 10 \times 2}{10^5} = 20000 \text{ QPS}$$

$$\text{Write QPS (лайки)} = \frac{100 \times 10^6 \times 2 \times 2}{10^5} = 4000 \text{ QPS}$$

$$\text{Write QPS (посты)} = \frac{100 \times 10^6 \times 0.1}{10^5} = 100 \text{ QPS}$$

TPS (Transactions Per Second) — транзакции с гарантиями ACID:

- Публикация поста требует транзакции (INSERT в posts + UPDATE user stats)

³Internet of Things — сетевое взаимодействие физических устройств с датчиками и контроллерами. Характеризуется высокой частотой передачи небольших порций данных (телеметрия, метрики).

⁴Структурированная информация, описывающая свойства других данных: время создания, размер, тип, владелец, права доступа. Используется для индексирования и маршрутизации без чтения основного содержимого.

⁵Логический контекст взаимодействия клиента с системой между множественными запросами. Хранит состояние пользователя: аутентификацию, корзину, настройки. Реализуется через cookies + серверное хранилище (Redis) или JWT-токены.

⁶Service Level Agreement — соглашение об уровне обслуживания с количественными метриками: uptime (99.9% = 8.76 ч простоя/год), задержка (p99 < 200 мс).

$$\text{TPS} \approx 100 \text{ TPS}$$

IOPS (I/O Operations Per Second) — операции диска:

- Загрузка изображений: запись на диск/object storage

$$\text{IOPS (изображения)} = \frac{100 \times 10^6 \times 5}{10^5} = 5000 \text{ IOPS}$$

Пиковая нагрузка: обычно в 3-5 раз выше средней (вечерние часы, выходные).

2.2.1.5. Оценка объёма хранения

Пример — хранение постов за год:

- 100 QPS записи $\times$ 30M секунд/год = 3 млрд постов
- Средний размер поста: 1 КБ текста + 500 байт метаданных

$$\text{Объём} = 3 \times 10^9 \times 1.5 \text{ КБ} \approx 4.5 \text{ ТБ/год}$$

С учётом индексов и репликации ($\times 3$): 15 ТБ/год.

2.2.1.6. Иерархия памяти

Уровень	Латентность	Пропускная способность
L1 cache	1 нс	1 ТБ/с
L2 cache	4 нс	500 ГБ/с
L3 cache	12 нс	200 ГБ/с
RAM	100 нс	50 ГБ/с
SSD (random)	100 мкс	500 МБ/с
SSD (seq)	50 мкс	3 ГБ/с
HDD (random)	10 мс	1 МБ/с
HDD (seq)	5 мс	200 МБ/с

Случайный доступ на HDD в 200 раз дороже последовательного. На SSD разница меньше (5-10х), но всё ещё значительна.

2.2.1.7. Фундаментальные компромиссы

Read vs Write: оптимизация под чтение замедляет запись и наоборот.

Space vs Time:

- Компрессия** — тратим CPU за уменьшение места на диске
- Денормализация** — тратим место за скорость чтения
- Индексы** — тратим место и замедляем запись за ускорение чтения

2.2.2. Лекция 6: LSM Дерево

2.2.2.1. Общая идея

LSM-дерево решает проблему эффективной записи путём отложенной сортировки и слияния. Вместо немедленной записи на диск в отсортированную структуру, данные буферизуются в памяти и периодически сбрасываются отсортированными порциями.

2.2.2.2. Отсортированная таблица строк (Sorted String Table, SSTable)

SSTable — иммутабельный файл с парами ключ-значение, отсортированными по ключу.

Внутренняя структура файла (последовательно):

[Data Block 1][Data Block 2]...[Data Block N][Index Block][Meta Block][Footer]

Data Block:

- Записи: [key_len][key][value_len][value] отсортированные по ключу
- Restart points: массив смещений для быстрого поиска внутри блока (каждые 16 записей)
- Block trailer: [compression_type][crc32]

Index Block:

- Записи формата: [last_key_in_block][block_offset][block_size]
- Позволяет определить нужный блок без чтения всех данных

Meta Block:

- Bloom Filter для всего файла
- Статистика: min/max ключи, количество записей, histogram распределения

Footer (фиксированный размер 48 байт):

- Смещение и размер Index Block
- Смещение и размер Meta Block
- Magic number для валидации формата

Поиск в SSTable — $\mathcal{O}(\log b + k)$, b — количество блоков, k — размер блока

- Бинарный поиск по индексу блоков $\mathcal{O}(\log b)$
- Линейное сканирование внутри блока $\mathcal{O}(k)$

2.2.2.3. Memtable

Memtable — изменяемая структура в памяти для накопления записей перед сбросом в SSTable.

Внутренняя структура — сбалансированное дерево (Red-Black Tree или Skip List):

- Поддерживает отсортированный порядок ключей
- При достижении порога размера сбрасывается в новый SSTable
- Вставка в Memtable — $\mathcal{O}(\log m)$, m — количество ключей в Memtable
- Конвертация в SSTable — $\mathcal{O}(m)$, последовательная запись отсортированных данных

Skip List предпочтительнее Red-Black Tree для Memtable поскольку не требует ротаций поддеревьев при вставке

Сложность операций Skip List:

- Insert/Delete/Search — $\mathcal{O}(\log n)$ в среднем, $\mathcal{O}(n)$ в худшем (крайне редко)

2.2.2.4. Bloom Filter

Bloom Filter — вероятностная структура для быстрой проверки отсутствия ключа в SSTable.

Главная идея: избежать дорогих дисковых операций для поиска несуществующих ключей. Вместо чтения полностью всех SSTable с диска, сначала проверяется Bloom Filter в памяти — если он говорит “ключа нет”, SSTable гарантированно не содержит ключ.

Интеграция в SSTable:

- Bloom Filter создаётся при формировании SSTable из Memtable
- Хранится в Meta Block SSTable файла
- При открытии SSTable загружается в память (обычно 10 бит на ключ)
- Проверяется перед любым обращением к данным SSTable

Внутренняя структура:

```
BitArray[m] = [0|1|0|1|1|0|1|0|...] // m бит
HashFunctions[k] = [h1, h2, ..., hk] // k независимых хеш-функций
```

Параметры:

- m — размер битового массива
- k — количество хеш-функций
- n — ожидаемое количество элементов

Алгоритм добавления элемента x:

```
for i = 1 to k:
    index = hi(x) mod m
    BitArray[index] = 1
```

Алгоритм проверки элемента x:

```
for i = 1 to k:
    index = hi(x) mod m
    if BitArray[index] == 0:
        return NOT_PRESENT // гарантированно отсутствует
return MAYBE_PRESENT // возможно присутствует
```

- Проверка ключа — $\mathcal{O}(k)$, k — количество хеш-функций (обычно 3-10)
- Добавление ключа — $\mathcal{O}(k)$
- False positive rate: $(1 - e^{\{-k \frac{n}{m}\}})^k$ при случайном распределении
- False negative rate: 0 (если Bloom Filter говорит “нет”, ключа точно нет)
- Память: m бит независимо от размера ключей

Экономия ресурсов:

- Без Bloom Filter: чтение Index Block + Data Block для каждого отсутствующего ключа
- С Bloom Filter: только проверка в памяти для 99%+ отсутствующих ключей (при p=1%)

2.2.2.5. Компакция – причины, уровни

Компакция объединяет несколько SSTable файлов в один для:

- Уменьшения числа файлов (улучшение производительности чтения)
- Удаления устаревших версий данных (delete операции, перезаписи)
- Уменьшения размера базы данных (удаление дубликатов)

Концепция уровней: LSM использует иерархию уровней, каждый уровень содержит отсортированные данные:

- L_0 : несортированные SSTable, пришедшие из Memtable (ключи **пересекаются**)
- L_1 : отсортированные SSTable, ключи **не пересекаются** внутри уровня
- $L_2, L_3, \dots, L_n$: каждый уровень в 10 раз больше предыдущего

Размер уровней:

- L_0 : до 4 файлов (типичный порог), каждый 2 МБ
- L_1 : 20 МБ ($10 \times$ размер L_0)
- L_i : 10^i МБ

2.2.2.6. Критерии запуска компакций

Компакция запускается когда:

1. L_0 достигает порога (обычно 4 файла) $\rightarrow$ компакция $L_0 \rightarrow L_1$
2. Размер уровня L_i превышает лимит $\rightarrow$ компакция $L_i \rightarrow L_{i+1}$
3. Поиск затрагивает слишком много файлов на одном уровне $\rightarrow$ принудительная компакция

2.2.2.7. Что происходит при компакциях

Компакция $L_0 \rightarrow L_1$:

- Читаем несортированные файлы из L_0 (возможны пересечения ключей)
- Читаем перекрывающиеся данные из L_1 (может быть много файлов)
- Выполняем k-way merge (слияние k отсортированных последовательностей)
- Удаляем старые версии данных во время слияния
- Записываем результат в L_1
- Сложность: $\mathcal{O}(N)$ где N — размер всех данных, значительная Ввод-вывод⁷ нагрузка

Компакция $L_i \rightarrow L_{i+1}$ ($i \geq 1$):

- Выбираем файлы из L_i , пересекающиеся по ключам с L_{i+1}
- Выполняем 2-way merge: отсортированные данные из L_i + перекрывающиеся из L_{i+1}
- Записываем результат в L_{i+1}
- **Сложность: $\mathcal{O}(S)$ где S — размер обрабатываемых данных (очень эффективно, т.к. ключи отсортированы)**

2.2.2.8. Почему L_0 специальный?

L_0 — единственный уровень, где ключи **пересекаются**. Это делает компакцию L_0 дорогой:

- Нужно проверить все файлы L_0 друг против друга
- Нужно слить с **потенциально всеми** файлами L_1 (так как ключи могут быть везде)
- Поэтому ограничивают число файлов в L_0 (например, max 4 файла)

На уровнях L_1 и выше ключи не пересекаются внутри уровня, что позволяет эффективно сливать только перекрывающиеся по ключам диапазоны.

2.2.2.9. Tombstones

Tombstone — специальная запись-маркер удаления, так как SSTable иммутабельны.

- Delete — $\mathcal{O}(\log m)$, запись tombstone в Memtable
- Физическое удаление происходит при компакции, когда tombstone достигает последнего уровня

2.2.2.10. Полная структура LSM-дерева

Архитектура компонентов:

1. WAL для восстановления Memtable после сбоя
2. Активный Memtable для записи
3. Иммутабельные Memtable, ожидающие сброса
4. Множественные уровни SSTable с Bloom Filter для каждого

CRUD операции:

- Create (вставка новой записи) — $\mathcal{O}(\log m)$, m — размер Memtable
 - Запись в WAL $\mathcal{O}(1)$ — последовательная запись в конец файла
 - Вставка в Memtable (Skip List) $\mathcal{O}(\log m)$
 - При переполнении Memtable становится иммутабельным, создаётся новый
- Read (поиск записи) — $\mathcal{O}(\log m + L(k + \log b))$, L — количество уровней, m — размер Memtable, b — кол-во блоков в SSTable
 - Поиск в активном Memtable $\mathcal{O}(\log m)$
 - Поиск в иммутабельных Memtable $\mathcal{O}(\log m)$ для каждого
 - Для каждого уровня SSTable:
 - Проверка Bloom Filter $\mathcal{O}(k)$, k — количество хеш-функций
 - Если Bloom положительный: бинарный поиск в Index Block $\mathcal{O}(\log b)$ + чтение Data Block
 - Возврат первого найденного значения (свежие данные приоритетнее)

⁷Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

- Update (обновление записи) – $\mathcal{O}(\log m)$
 - Идентично Create — новая версия записывается в Memtable
 - Старые версии остаются в SSTable до компакций
 - При чтении возвращается самая свежая версия
- Delete (удаление записи) – $\mathcal{O}(\log m)$
 - Запись tombstone в WAL $\mathcal{O}(1)$
 - Вставка tombstone в Memtable $\mathcal{O}(\log m)$
 - Физическое удаление при компакции когда tombstone достигает последнего уровня
- Range Query (диапазонный запрос) – $\mathcal{O}(\log m + L \cdot \log b + R)$, R — размер результата
 - Создание итераторов для Memtable и каждого SSTable
 - Merge итераторов с учётом версионности
 - SSTable уже отсортированы — эффективное слияние

2.2.3. Лекция 7: В-дерево: индексирование на диске

В-дерево решает проблему эффективного использования кэша диска и минимизации случайных обращений путём группировки данных в узлы фиксированного размера, соответствующие блокам диска.

2.2.3.1. В-фактор и структура узла

В-фактор — это максимальное количество указателей на детей в одном узле (или эквивалентно, максимальное количество ключей + 1).

Внутренняя структура узла порядка B :

- Ключи: $[k_1, k_2, \dots, k_{B-1}]$ в отсортированном порядке (максимум $B - 1$ ключей)
- Указатели на детей: $[p_0, p_1, \dots, p_B]$ где p_i указывает на поддерево с ключами в диапазоне $[k_i, k_{i+1})$
- Листовой узел содержит значения вместо указателей на поддеревья
- Каждый узел занимает ровно одно дисковое обращение — это ключевая идея

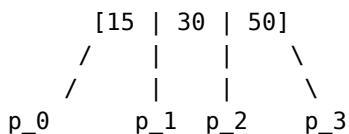
Выбор В-фактора:

- В выбирается так, чтобы узел точно поместился в один блок диска
- Если блок диска = 4 КБ, а каждая пара (ключ, указатель) занимает 10 байт, то $B \approx 400 - 500$
- Маленький B (например, $B=4$): много уровней дерева, много обращений к диску
- Большой B (например, $B=500$): глубина дерева $\mathcal{O}(\log_B n)$ очень мала (для 1 млн записей при $B=500$ глубина всего 3-4 уровня), но поиск внутри узла медленнее

Инвариант В-дерева:

- Все листья находятся на одной глубине h (в отличие от AVL, где глубина может отличаться на 1)
- Каждый внутренний узел содержит от $\lceil \frac{B}{2} \rceil$ до $B - 1$ ключей
- Корень содержит минимум 1 ключ

Пример узла В-дерева при $B=4$:



p_0: ключи < 15
p_1: ключи в [15, 30)
p_2: ключи в [30, 50)
p_3: ключи >= 50

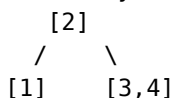
Пример роста В-дерева порядка $B=3$ при вставке 1,2,3,4,5:

Вставляем 1: [1]

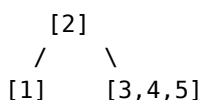
Вставляем 2: [1,2]

Вставляем 3: [1,2,3]

Вставляем 4: узел переполнен! Разделяем:



Вставляем 5:



При вставке нового элемента в переполненный узел:

- Узел разделяется на две половины
- Медиана поднимается в родителя

- Если родитель тоже переполнен, рекурсивно разделяем его
- Это может привести к вырастанию корня и увеличению высоты дерева на 1
- Create – $\mathcal{O}(\log_B n)$
 - Поиск листового узла $\mathcal{O}(\log_B n)$ дисковых обращений
 - Вставка ключа-значения в лист $\mathcal{O}(1)$
 - При переполнении узла: разделение на две половины и поднятие медианы $\mathcal{O}(\log_B n)$ обращений вверх по дереву
- Read – $\mathcal{O}(\log_B n)$
 - Бинарный поиск по ключам текущего узла $\mathcal{O}(\log B)$ — в памяти, быстро
 - Переход к дочернему узлу через одно дисковое обращение
 - Повторение до листа: максимум $\mathcal{O}(\log_B n)$ обращений
- Update – $\mathcal{O}(\log_B n)$
 - Поиск записи $\mathcal{O}(\log_B n)$
 - Обновление значения in-place (B-дерево не версионировано, в отличие от LSM)
- Delete – $\mathcal{O}(\log_B n)$
 - Поиск и удаление ключа $\mathcal{O}(\log_B n)$
 - При недостаточном количестве ключей: слияние с соседним узлом или перемещение ключей из соседа
- Range Query – $\mathcal{O}(\log_B n + \frac{R}{B})$, R — размер результата
 - Поиск левой границы $\mathcal{O}(\log_B n)$
 - Последовательное сканирование листьев (один узел = один читаемый блок диска)
 - Эффективно благодаря локальности: листья часто находятся рядом на диске

Преимущества относительно LSM:

- Нет компакций, стабильная задержка
- Меньше I/O амортизировано: B больше, чем типичный размер блока

Недостатки:

- Запись требует read-modify-write: чтение узла → изменение → запись обратно
- Сложность балансировки при вставке и удалении
- Для частых записей LSM может затратить сильно меньше Ввод-вывод⁸

⁸Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

2.2.4. Лекция 8: ACID, уровни изоляции, аналитические и транзакционные хранилища

2.2.4.1. Транзакции

Транзакция — Последовательность операций чтения и записи в базе данных, рассматриваемая как единая логическая единица работы. Гарантирует атомарность выполнения: либо все операции завершаются успешно и фиксируются (COMMIT), либо при ошибке все изменения откатываются (ROLLBACK).

```
BEGIN;  
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
COMMIT; -- или ROLLBACK для отката
```

2.2.4.2. ACID (Atomicity, Consistency, Isolation, Durability)

- **Atomicity** — транзакция выполняется полностью или не выполняется вовсе; реализуется через WAL
- **Consistency** — поддерживаются ограничения целостности (foreign keys, constraints)
- **Isolation** — параллельные транзакции не влияют друг на друга; регулируется уровнями изоляции
- **Durability** — изменения сохраняются навсегда после COMMIT; обеспечивается fsync

2.2.4.3. Транзакционные и аналитические хранилища

OLTP — Online Transaction Processing — класс систем обработки транзакций в реальном времени. Характеризуется большим количеством коротких транзакций (чтение, вставка, обновление, удаление), строковым хранением данных и требованиями к консистентности.

- Паттерн: много мелких операций чтения/записи отдельных записей
- Примеры запросов:
 - SELECT * FROM users WHERE id = 123
 - UPDATE orders SET status = 'shipped' WHERE id = 456
 - INSERT INTO transactions (user_id, amount) VALUES (789, 100.50)
- Строковое хранение (row-oriented): вся строка читается за один блок диска
- СУБД⁹: PostgreSQL, MySQL, Oracle
- Применение: интернет-магазины, банковские системы, CRM¹⁰

OLAP — Online Analytical Processing — класс систем для аналитической обработки данных. Ориентирован на выполнение сложных агрегирующих запросов по большим объёмам данных, использует колоночное хранение для оптимизации производительности аналитики.

- Паттерн: большие агрегирующие запросы по миллионам строк
- Примеры запросов:
 - SELECT country, SUM(revenue) FROM sales GROUP BY country
 - SELECT date, COUNT(*) FROM events WHERE timestamp > '2024-01-01' GROUP BY date
 - Анализ трендов, построение отчётов, data science
- Колоночное хранение (column-oriented): колонки хранятся последовательно
 - Читаем только нужные колонки (не всю строку)
 - Лучше сжатие: однотипные данные в одном блоке
 - Векторизованные вычисления на CPU
- СУБД¹¹: ClickHouse, Apache Druid, Vertica, Snowflake
- Применение: аналитика метрик, BI¹²-дашборды, Data Warehouses
- Недостатки: медленные вставки и UPDATE/DELETE отдельных записей

⁹Система управления базами данных — комплекс программных средств для создания, модификации и управления данными в базе данных. Обеспечивает структурированное хранение, эффективный доступ, целостность данных и управление конкурентным доступом.

¹⁰Customer Relationship Management — система управления взаимоотношениями с клиентами. Программное обеспечение для автоматизации стратегий взаимодействия с клиентами, включая продажи, маркетинг, техподдержку и аналитику клиентских данных.

¹¹Система управления базами данных — комплекс программных средств для создания, модификации и управления данными в базе данных. Обеспечивает структурированное хранение, эффективный доступ, целостность данных и управление конкурентным доступом.

2.2.4.4. Сжатие колонок

- **Run-Length Encoding** — $[5, 5, 5, 3, 3, 7] \rightarrow [(5, 4), (3, 2), (7, 1)]$
- **Dictionary Encoding** — словарь уникальных значений + индексы
- **Bit Packing** — минимум битов на число
- **Delta Encoding** — разности между значениями для timestamp, id
- **Frame of Reference** — вычитание минимума + bit packing

¹²Business Intelligence — технологии и практики сбора, интеграции, анализа и представления бизнес-информации. Включает инструменты для создания отчётов, дашбордов, визуализации данных и поддержки принятия управленческих решений на основе данных.

2.2.5. Лекция 9: Хранилища в оперативной памяти

2.2.5.1. Аналитика: DuckDB и векторизация

Векторизованное выполнение — обработка батчами 1000-10000 строк вместо по одной:

- SIMD инструкции CPU (Single Instruction Multiple Data)
- Лучшее утилизация кэш-линий процессора
- Меньше overhead на интерпретацию

DuckDB — встраиваемая аналитическая СУБД¹³:

```
duckdb.sql("SELECT * FROM 'data.parquet' WHERE price > 100")
```

- Чтение Parquet, CSV, JSON без импорта
- Применение: ETL, data science, аналитика

2.2.5.2. Кэширование: Redis

Хранилище в оперативной памяти (in-memory key-value store).

Архитектура:

- Однопоточная модель + event-driven I/O (epoll/kqueue)
- Нет overhead на синхронизацию

Структуры данных:

- **Skip List** (sorted sets) — уже рассмотрен в LSM; `ZADD leaderboard 100 "player1"`
- **Incremental Rehashing** — постепенное перехэширование без блокировки
- **HyperLogLog** — вероятностный подсчёт уникальных; 12 КБ для миллиардов элементов
- Strings, Lists, Sets, Hashes, Bitmaps, Streams

Применение: кэширование, Сессия¹⁴, очереди, real-time аналитика

¹³Система управления базами данных — комплекс программных средств для создания, модификации и управления данными в базе данных. Обеспечивает структурированное хранение, эффективный доступ, целостность данных и управление конкурентным доступом.

¹⁴Логический контекст взаимодействия клиента с системой между множественными запросами. Хранит состояние пользователя: аутентификацию, корзину, настройки. Реализуется через cookies + серверное хранилище (Redis) или JWT-токены.

2.3. Модуль 3: Методы поиска похожих/близких элементов

2.3.1. Лекция 10: Специализированные индексы для поиска

2.3.1.1. Inverted Index (перевёрнутые индексы)

- Постинг-листы (posting lists)
- Применение: Elasticsearch, Lucene

2.3.1.2. Trie (префиксные деревья)

- Автодополнение, префиксный поиск

2.3.1.3. Bitmap indexes

- Для категориальных данных

2.3.2. Лекция 11: Векторные индексы для поиска подобия

2.3.2.1. Основы векторных представлений

2.3.2.2. HNSW (Hierarchical Navigable Small World)

- Граф как индекс
- Поиск соседей

2.3.2.3. LSH (Locality-Sensitive Hashing)

- Вероятностный поиск

2.3.2.4. Примеры: Weaviate, Pinecone, Qdrant

2.3.3. Лекция 12: Геопространственные структуры

2.3.3.1. R-Tree

- Bounding boxes, MBR
- Применение: PostGIS

2.3.3.2. QuadTree и KD-Tree

- Разбиение пространства
- Когда какое дерево эффективнее

2.4. Модуль 4: Операционные компоненты

2.4.1. Лекция 13: Прокси и Rate Limiting

2.4.1.1. Forward Proxy / Reverse Proxy

2.4.1.2. Распределение нагрузки

- Round Robin
- Least Connections
- Weighted Least Connections
- IP Hash
- Consistent Hashing

2.4.1.3. Ограничение частоты запросов

- Token Bucket
- Sliding Window

2.4.2. Лекция 14: Безопасность и управление доступом

2.4.2.1. Аутентификация и авторизация

2.4.2.2. RBAC и ABAC

2.4.2.3. Single Sign-On

2.4.3. Лекция 15: Логирование, мониторинг и планирование задач

2.4.3.1. Сбор логов

2.4.3.2. Сбор метрик (Prometheus)

2.4.3.3. Мониторинг и диагностика (Grafana)

2.4.3.4. Планировщики (Celery, Airflow, Dagster)

2.5. Модуль 5: System Design

2.5.1. Лекция 16: Разбор дизайна конкретной системы

2.5.1.1. Применение всех концепций на практике

3. Распределённые системы обработки данных

3.1. Модуль 1: Компоненты распределённых систем

3.1.1. Лекция 1: Основы распределённых систем

3.1.1.1. CAP-теорема

- Следствия компромиссов между согласованностью, доступностью и устойчивостью к разделению сети
- Примеры CA, AP, CP систем

3.1.1.2. Механизмы репликации данных

- Архитектура “ведущий-подчинённый” (Leader-Follower)
- Репликация с использованием кворумов (quorum-based replication)

3.1.1.3. ZooKeeper: распределённое решение проблем согласованности и отказоустойчивости

- Механизмы решения классических проблем распределённых систем
- Применение в системах Kafka и ClickHouse

3.1.2. Лекция 2: Гарантии консистентности и координация распределённых транзакций

3.1.2.1. Алгоритмы достижения консенсуса

- Raft
- Paxos

3.1.2.2. Модель конечной консистентности (Eventual consistency) и требование идемпотентности операций

3.1.2.3. Протокол двухфазного коммита (Two-Phase Commit, 2PC)

- Этапы выполнения
- Гарантии надёжности

3.2. Модуль 2: Распределённые сервисы

3.2.1. Лекция 3: Асинхронное взаимодействие сервисов

3.2.1.1. Event Bus и Event-Driven Architecture

3.2.1.2. Publish-Subscribe vs Message Queue

3.2.1.3. Saga Pattern

3.2.2. Лекция 4: Kafka

3.2.2.1. Внутреннее устройство

3.2.3. Лекция 5: Kubernetes

3.2.3.1. Роль и место в современных архитектурах

3.2.3.2. Области использования

3.2.3.3. Внутреннее устройство

3.2.4. Лекция 6: ClickHouse

3.2.4.1. Архитектура системы хранения и обработки данных

- Принципы организации хранения
- Методы компрессии данных

3.2.4.2. Партиционирование

- Выбор ключа
- Управление партициями (добавление, удаление)

3.2.4.3. Таблицы серии MergeTree как основной механизм персистентности

- Базовая таблица MergeTree и её специализированные варианты
- ReplacingMergeTree для управления версионированием
- ReplicatedMergeTree для обеспечения отказоустойчивости

3.2.5. Лекция 7: Apache Spark

3.2.5.1. Роль и место в современных архитектурах

3.2.5.2. Области использования

3.2.5.3. Внутреннее устройство

3.2.5.4. API

- Dataframe
- SparkSQL

3.2.6. Лекция 8: Erlang

3.2.6.1. Роль и место в современных архитектурах

3.2.6.2. Erlang VM (BEAM) и процессы

3.2.6.3. Модель акторов

3.2.6.4. Fault tolerance и “Let it crash” философия

3.2.6.5. OTP фреймворк (Supervisor, GenServer)

3.2.6.6. Синхронизация и распределённые вычисления

3.2.6.7. Elixir

3.3. Модуль 3: Распределенные БД

3.3.1. Лекция 9: Большие данные

3.3.1.1. Свойства

- Объем
- Скорость
- Разнообразие
- Достоверность

3.3.1.2. Архитектуры

- Modern Data Architecture
- Lambda
- Lakehouse

3.3.2. Лекция 10: Full Text Search в распределённых системах

3.3.2.1. Elasticsearch и Solr

- Sharding: hash-based, range-based
- Replication и replica factor
- Eventual consistency при индексировании

3.3.2.2. Распределённый поиск

- Broadcast query, merge результатов
- Search After вместо offset/limit

3.3.2.3. Масштабирование

- Hot shards проблема

- Index rollover, segment merging

3.3.3. Лекция 11: Документные хранилища

3.3.3.1. MongoDB: sharding strategies, write concerns

3.3.3.2. Object storage: MinIO

3.3.4. Лекция 12: Распределенные пространственные и графовые БД

3.3.4.1. Графовые БД

3.3.4.2. Пространственные БД

3.3.4.3. Шардирование графов и пространственных данных